

Partial Differential Equations 3 (Numerical)

Lecture 7 - Academic Year 2025/2026

Prof Halim Kusumaatmaja
Email: halim.kusumaatmaja@ed.ac.uk
Semester 2, Edinburgh Engineering

Aims and Objectives

1. To discuss implicit time and Crank-Nicolson schemes.
2. To consider the stability of Crank-Nicolson scheme.
3. To explain Thomas algorithm needed for the Crank-Nicolson scheme.

Implicit Time and Crank-Nicolson

In Lecture 5, when we discussed the explicit time scheme, we discretise the 1D heat conduction equation as

$$T_j^{n+1} - T_j^n = r_x(T_{j+1}^n - 2T_j^n + T_{j-1}^n), \quad (1)$$

where $r_x = \frac{\kappa \Delta t}{\Delta x^2}$. A better approach would be to include information from the new time-level $n + 1$. We could use the following

$$T_j^{n+1} - T_j^n = r_x(1 - \alpha)(T_{j+1}^n - 2T_j^n + T_{j-1}^n) + r_x\alpha(T_{j+1}^{n+1} - 2T_j^{n+1} + T_{j-1}^{n+1}), \quad (2)$$

where the coefficient α allows us to weigh the contributions from n and $n + 1$ time-levels. If we choose $\alpha = 1$, we have a fully implicit scheme:

$$T_j^{n+1} - T_j^n = r_x(T_{j+1}^{n+1} - 2T_j^{n+1} + T_{j-1}^{n+1}). \quad (3)$$

Choosing $\alpha = 0.5$ gives us the well known Crank-Nicolson time marching method:

$$\begin{aligned} T_j^{n+1} - T_j^n &= r_x/2(T_{j+1}^n - 2T_j^n + T_{j-1}^n) + r_x/2(T_{j+1}^{n+1} - 2T_j^{n+1} + T_{j-1}^{n+1}), \\ -r_x T_{j-1}^{n+1} + (2 + 2r_x)T_j^{n+1} - r_x T_{j+1}^{n+1} &= r_x T_{j-1}^n + (2 - 2r_x)T_j^n + r_x T_{j+1}^n. \end{aligned} \quad (4)$$

The problem with implicit or Crank-Nicolson methods is that not all terms are known when we want to compute T_j^{n+1} . We know everything on the right hand side of Eq. (4), but not on the left hand side. This means we end up with a system of simultaneous equations that is more cumbersome to solve. As we have learned in Lecture 4, it is advantageous to write such system of simultaneous of equations as a matrix equation. Before we can do that, however, we need to discuss the boundary conditions at the most left and right grid points in our computational domain. If we focus on Dirichlet boundary conditions, we have

$$\text{left :} \quad (2 + 2r_x)T_j^{n+1} - r_x T_{j+1}^{n+1} = r_x T_{j-1}^n + (2 - 2r_x)T_j^n + r_x T_{j+1}^n + r_x T_{j-1}^{n+1}, \quad (5)$$

$$\text{right :} \quad -r_x T_{j-1}^{n+1} + (2 + 2r_x)T_j^{n+1} = r_x T_{j-1}^n + (2 - 2r_x)T_j^n + r_x T_{j+1}^n + r_x T_{j+1}^{n+1}. \quad (6)$$

The terms highlighted in blue are known as part of the specified Dirichlet boundary conditions.

Exercise 1: How does the Crank-Nicolson scheme get modified if we have a Neumann boundary condition instead?

Given the Dirichlet boundary condition, we can rewrite the Crank-Nicolson scheme using a matrix equation

$$\begin{bmatrix} 2+2r_x & -r_x & & & \\ -r_x & 2+2r_x & -r_x & & \\ & -r_x & 2+2r_x & -r_x & \\ & & \dots & & \\ & & -r_x & 2+2r_x & -r_x \\ & & & -r_x & 2+2r_x & -r_x \\ & & & & -r_x & 2+2r_x \end{bmatrix} \begin{bmatrix} T_1^{n+1} \\ T_2^{n+1} \\ T_3^{n+1} \\ \dots \\ T_{n_x-4}^{n+1} \\ T_{n_x-3}^{n+1} \\ T_{n_x-2}^{n+1} \end{bmatrix} = \begin{bmatrix} d_1^n \\ d_2^n \\ d_3^n \\ \dots \\ d_{n_x-4}^n \\ d_{n_x-3}^n \\ d_{n_x-2}^n \end{bmatrix} \quad (7)$$

with

$$\begin{aligned} j=1 & \rightarrow d_j^n = r_x T_{j-1}^n + (2-2r_x)T_j^n + r_x T_{j+1}^n + r_x T_{j-1}^{n+1}, \\ j=n_x-2 & \rightarrow d_j^n = r_x T_{j-1}^n + (2-2r_x)T_j^n + r_x T_{j+1}^n + r_x T_{j+1}^{n+1}, \\ \text{other } j's & \rightarrow d_j^n = r_x T_{j-1}^n + (2-2r_x)T_j^n + r_x T_{j+1}^n. \end{aligned}$$

If preferred, the d^n terms can be written as a matrix as well, but the key point is that they are all known at the time of computation. To avoid confusion, our notation is that the grid point index runs from $T_0, T_1, \dots, T_{n_x-2}, T_{n_x-1}$ and we have assigned T_0 and T_{n_x-1} as the boundary nodes with known values.

von Neumann Stability Analysis

The Crank-Nicolson scheme is more complex, and so it is worth understanding why it is so popular and why it is worth the effort. To do this we will go back to the von Neumann stability analysis we did in Lecture 5. Recall that we consider the error term and expand it in Fourier series,

$$\epsilon(x, t) = \sum_{m=-M}^M E_m(t) e^{ik_m x}. \quad (8)$$

The meaning of the terms are as before. Considering each mode, for the Crank-Nicolson scheme, we have

$$\begin{aligned} -r_x \epsilon_{j-1}^{n+1} + (2+2r_x) \epsilon_j^{n+1} - r_x \epsilon_{j+1}^{n+1} &= r_x \epsilon_{j-1}^n + (2-2r_x) \epsilon_j^n + r_x \epsilon_{j+1}^n \\ E_m(t + \Delta t) e^{ik_m x} \left[-r_x e^{-ik_m \Delta x} + (2+2r_x) - r_x e^{ik_m \Delta x} \right] &= E_m(t) e^{ik_m x} \left[r_x e^{-ik_m \Delta x} + (2-2r_x) + r_x e^{ik_m \Delta x} \right] \end{aligned}$$

This allows us to consider how the error term grows with time

$$\begin{aligned} g &= \frac{E_m(t + \Delta t)}{E_m(t)} = \frac{r_x e^{-ik_m \Delta x} + (2-2r_x) + r_x e^{ik_m \Delta x}}{-r_x e^{-ik_m \Delta x} + (2+2r_x) - r_x e^{ik_m \Delta x}}, \\ g &= \frac{2 + r_x(-2 + e^{ik_m \Delta x} + e^{-ik_m \Delta x})}{2 + r_x(2 - e^{ik_m \Delta x} - e^{-ik_m \Delta x})} = \frac{2 + 2r_x(-1 + \cos k_m \Delta x)}{2 + 2r_x(1 - \cos k_m \Delta x)}, \\ g &= \frac{1 - 2r_x \sin^2(k_m \Delta x/2)}{1 + 2r_x \sin^2(k_m \Delta x/2)}. \end{aligned} \quad (9)$$

Since the $\sin^2$ term is always a positive number, we can conclude that $|g| \leq 1$, which means the Crank-Nicolson scheme is unconditionally stable. This is a useful property of any numerical scheme.

At this point, look at the interactive examples on the weblink (see link below). The description of the example problem is the same as in Lecture 5. Consider choices of Δt and Δx which are unstable for the FTCS scheme in Lecture 5. Are they stable in Crank-Nicolson scheme?

Exercise 2: Derive the stability criterion for the fully implicit scheme.

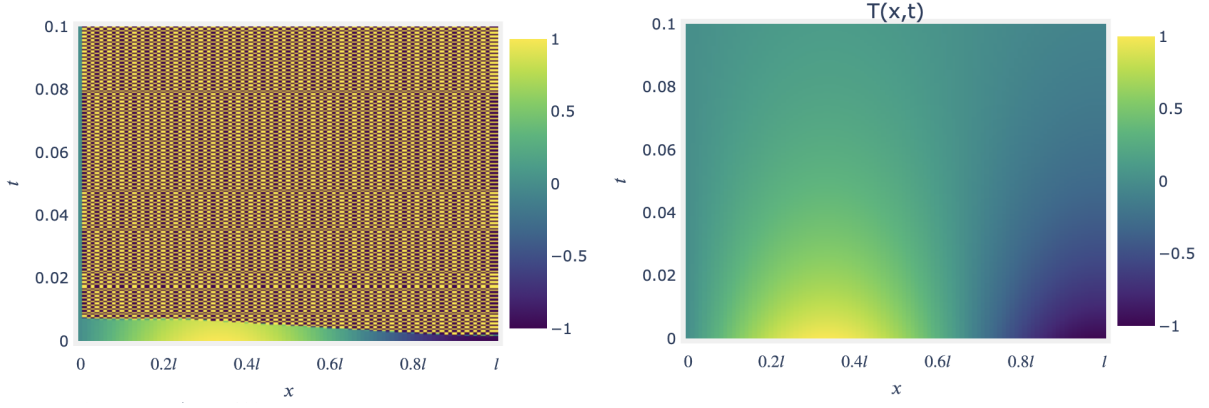


Figure 1: (Left) Example solution that is unstable using the FTCS method with $\Delta x = 0.01$ and $\Delta t = 6 \times 10^{-5}$. (Right) The same parameter is stable when using Crank-Nicolson method.

Thomas Algorithm

The interactive tool uses the so-called Thomas algorithm to solve the matrix equation. This is particularly effective when we have a tri-diagonal matrix, that is a matrix where all the elements are zero except for those in the main diagonal, the subdiagonal/lower diagonal (the first diagonal below), and the supradiagonal/upper diagonal (the first diagonal above). Upon inspection this is the situation we have in Eq. (7). The key idea behind the Thomas algorithm is to turn the matrix equation in Eq. (7), rewritten as follows for convenience,

$$\begin{bmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & a_3 & b_3 & c_3 & \\ & & & \dots & \\ & & & & a_{n_x-4} & b_{n_x-4} & c_{n_x-4} & 0 \\ & & & & & a_{n_x-3} & b_{n_x-3} & c_{n_x-3} \\ & & & & & & a_{n_x-2} & b_{n_x-2} \end{bmatrix} \begin{bmatrix} T_1^{n+1} \\ T_2^{n+1} \\ T_3^{n+1} \\ \dots \\ T_{n_x-4}^{n+1} \\ T_{n_x-3}^{n+1} \\ T_{n_x-2}^{n+1} \end{bmatrix} = \begin{bmatrix} d_1^n \\ d_2^n \\ d_3^n \\ \dots \\ d_{n_x-4}^n \\ d_{n_x-3}^n \\ d_{n_x-2}^n \end{bmatrix} \quad (10)$$

to a form that looks like

$$\begin{bmatrix} 1 & \beta_1 & & & \\ 0 & 1 & \beta_2 & & \\ & 0 & 1 & \beta_3 & \\ & & & \dots & \\ & & & & 0 & 1 & \beta_{n_x-4} \\ & & & & & 0 & 1 & \beta_{n_x-3} \\ & & & & & & 0 & 1 \end{bmatrix} \begin{bmatrix} T_1^{n+1} \\ T_2^{n+1} \\ T_3^{n+1} \\ \dots \\ T_{n_x-4}^{n+1} \\ T_{n_x-3}^{n+1} \\ T_{n_x-2}^{n+1} \end{bmatrix} = \begin{bmatrix} f_1^n \\ f_2^n \\ f_3^n \\ \dots \\ f_{n_x-4}^n \\ f_{n_x-3}^n \\ f_{n_x-2}^n \end{bmatrix}. \quad (11)$$

If we can achieve this, we can readily obtain the T^{n+1} values. More specifically, from the matrix equation, we know

$$T_{n_x-2}^{n+1} = f_{n_x-2}^n. \quad (12)$$

Then, we can recursively find $T_{n_x-3}^{n+1}$ all the way to T_1^{n+1} by solving the equation

$$\begin{aligned} T_j^{n+1} + \beta_j T_{j+1}^{n+1} &= f_j^n, \\ T_j^{n+1} &= f_j^n - \beta_j T_{j+1}^{n+1}. \end{aligned} \quad (13)$$

Every term on the RHS of the above equation is known.

We are now left to work out how to manipulate Eq. (10) to Eq. (11). For the first row, we can see that we can achieve the transformation by

$$\beta_1 = \frac{c_1}{b_1}, \quad f_1^n = \frac{d_1^n}{b_1}. \quad (14)$$

For the second row, we can subtract

$$a_2 T_1^{n+1} + b_2 T_2^{n+1} + c_2 T_3^{n+1} = d_2^n$$

with the first row of the transformed matrix multiplied by a_2 :

$$a_2 \times (T_1^{n+1} + \beta_1 T_2^{n+1}) = a_2 \times (f_1^n),$$

to get

$$\begin{aligned} (b_2 - a_2 \beta_1) T_2^{n+1} + c_2 T_3^{n+1} &= d_2^n - a_2 f_1^n, \\ T_2^{n+1} + \frac{c_2}{b_2 - a_2 \beta_1} T_3^{n+1} &= \frac{d_2^n - a_2 f_1^n}{b_2 - a_2 \beta_1}. \end{aligned}$$

We can identify

$$\beta_2 = \frac{c_2}{b_2 - a_2 \beta_1}, \quad f_2^n = \frac{d_2^n - a_2 f_1^n}{b_2 - a_2 \beta_1}. \quad (15)$$

Following the steps, in general, we have

$$\beta_j = \frac{c_j}{b_j - a_j \beta_{j-1}}, \quad f_j^n = \frac{d_j^n - a_j f_{j-1}^n}{b_j - a_j \beta_{j-1}}. \quad (16)$$

Additional Materials

- Interactive tool: <https://kusumaatmaja.github.io/PDE3/examples>

Version Control:

- version 1.0 on 20 December 2025.